

From Script to Agent: A Bounded Reflection Loop and Structured-Output Contract for a LangGraph Research Assistant, Deployed to Azure

Kaustubha Venkata Eluri
Northeastern University

Abstract—A large language model wired to a web-search tool is not, by itself, an agent — it is a script that calls an API and then writes whatever it finds, with no mechanism to notice that a result set is thin. This paper documents the design and incremental construction of a LangGraph-based research assistant that adds a genuine decision point to that pipeline: after each search, a second LLM call judges whether the retrieved information is sufficient to write a solid summary, and the graph conditionally routes back to search again if it is not, bounded by a fixed retry cap to prevent runaway execution. The system additionally forces its final output through a Pydantic-validated schema rather than free text, isolates all language-model calls behind a single swappable client, and generates a cover image from a model-authored, vision-only prompt as a separate graph node. We describe the project’s phase-by-phase construction — from a bare LLM call, through manual tool calling, to a full LangGraph state machine with structured output and multimodal generation — and its containerized deployment to Azure Container Apps, Blob Storage, and Static Web Apps. We candidly report the concrete deployment failures encountered (a CPU-architecture mismatch between development and cloud, a malformed secret value, and a stale-tag caching bug) and the system’s limitations, since the project was built explicitly as a learning exercise in agentic design patterns rather than as a production research tool.

Index Terms—LLM agents, LangGraph, tool calling, reflection, structured output, multimodal generation, containerized deployment, Azure

I. INTRODUCTION

The term “AI agent” is used loosely enough in current practice that it is worth stating precisely what distinguishes an agent from a script that happens to call a language model. A script executes a fixed sequence of steps regardless of what it observes along the way: search, then summarize, always in that order, on whatever the search tool happened to return. An agent, by contrast, makes at least one decision about its own next step based on the state of the world it has observed so far — in the simplest useful case, a decision about whether it has done enough work yet to proceed, or whether it should do more of the same work before moving on.

This paper documents a small research-assistant application built specifically to explore that distinction in a minimal, legible way, rather than to solve a novel research problem. Given a topic, the system searches the live web for current information, and — critically — does not simply write a summary of whatever the first search returns. Instead, a second language-model call inspects the search results and renders a binary judgment, `SUFFICIENT` or `MORE`, and the underlying

state machine conditionally loops back to search again if the judgment is `MORE`, up to a fixed cap of three searches. Only once the reflection step is satisfied (or the cap is reached) does the system proceed to write a final report, and even then the report is not returned as a paragraph of free text: it is forced through a Pydantic schema with a fixed shape (headline, bullet points, sources, and a text-only image-generation prompt), and a separate graph node turns that last field into an actual cover image via an external image-generation API.

The system was deliberately built in incremental phases, each kept as a separate, runnable file, precisely so that each concept — a bare LLM call, manual tool calling, graph-based orchestration, conditional state routing, structured output, and multimodal generation — could be inspected and understood in isolation before being composed into the finished FastAPI-and-React application that was containerized and deployed to Azure. This paper documents that progression, the reflection-loop and structured-output design that we consider the project’s central contribution, the deployment path and the concrete failures encountered while shipping it to the cloud, and the system’s limitations.

II. RELATED WORK

Two established patterns underlie this system, both now common in LLM application engineering rather than novel to this project. The first is tool calling (function calling): rather than asking a language model to produce a fact directly — in this case, current information about a topic the model was not trained on — the model instead emits a request to invoke an external tool, here a Tavily web-search call, and the tool’s real result is fed back into the model’s context so that its subsequent text is grounded in that result. The second is graph- or state-machine-based agent orchestration, of which LangGraph is one widely used implementation: an agent’s control flow is expressed as a set of nodes and edges over an explicit state object, including conditional edges whose destination is chosen at runtime based on the current state, which is what allows a reflection-style loop (search, judge, conditionally search again) to be expressed directly in the orchestration layer rather than hand-rolled in application code.

A third pattern, structured output, constrains a language model’s final response to conform to a declared schema (here, a Pydantic model) rather than accepting an arbitrary string that a downstream consumer must then parse heuristically. This

```

React (frontend)
|
FastAPI (api.py)
|
LangGraph state machine
|
+-- search -----> Tavily web search
+-- analyze -----> Groq LLM (llama-3.3-70b)
| "enough info yet?"
| loops back to search if not,
| up to MAX_SEARCHES = 3
+-- summarize ----> Groq LLM, forced into
| a Pydantic schema
+-- generate_image -> Pollinations.ai
                        (free image generation)

```

Fig. 1. The LangGraph state machine underlying the agent. The analyze node is the reflection step: its conditional edge either routes back to search or forward to summarize, capped by MAX_SEARCHES.

matters specifically because the system’s consumer is a web frontend that needs a predictable headline field, a predictable list of bullet points, and a predictable list of sources — not a paragraph it must guess how to split.

This paper does not claim novelty in any of these three patterns individually. Its contribution is the concrete, incrementally documented composition of a bounded reflection loop, a structured-output contract, and a downstream multimodal-generation step into a single small agent, together with an honest account of what broke during its containerized cloud deployment.

III. SYSTEM ARCHITECTURE

A. Overview

The system is a LangGraph state machine wrapped in a FastAPI backend and served to a React/Vite frontend, containerized with Docker and deployed to Azure. Figure 1 (reproduced from the project’s own architecture documentation) shows the graph.

The graph exposes six nodes in total (search, analyze, summarize, generate_image, plus the entry and routing structure) and one conditional edge, which is the reflection-loop routing decision described in Section IV. All language-model calls in the graph — the analyze judgment and the summarize call — are issued through a single ChatGroq client instance (llama-3.3-70b-versatile), so that swapping the underlying model provider is a change to one wrapper rather than a change scattered across every node that happens to need a completion.

B. Incremental construction

Rather than writing the finished graph directly, the system was built as a sequence of eight increasingly capable, independently runnable phases, each retained in the repository rather than discarded once superseded:

- **Phase 1** (main.py) — a single, bare Groq LLM call, with no tools and no orchestration.
- **Phase 2** (phase2_tool_call.py) — manual tool calling: the LLM decides that a search is needed and

emits a tool-call request; application code executes the actual Tavily search and hands the result back; the LLM interprets it. No LangGraph is used yet — the loop is hand-written.

- **Phase 3** (phase3_langgraph.py) — the same tool-call loop is re-expressed using LangGraph, replacing the hand-written control flow with a declared graph.
- **Phase 4** (phase4_state_routing.py) — a custom state object and a conditional edge are introduced: this is the phase in which the SUFFICIENT/MORE reflection judgment and the search-again routing first appear.
- **Phase 5** (phase5_structured_output.py) — the final report is forced through a Pydantic ResearchReport model via llm.with_structured_output (ResearchReport), replacing a free-text final answer.
- **Phase 6** (phase6_image_generation.py) — an additional generate_image node is added, consuming the report’s image_prompt field; this is the version actually imported and run by the API layer.
- **Phase 7** (api.py) — a FastAPI wrapper exposes the Phase 6 graph over HTTP.
- **Phase 8** (frontend/) — a React/Vite single-page UI calls the FastAPI backend.

This phase-by-phase retention is itself a deliberate design choice, not an artifact of disorganization: it keeps each individual concept — bare completion, manual tool calling, graph-based orchestration, conditional routing, structured output, multimodal generation — separately inspectable, rather than folding all of them into one file where the contribution of any single pattern would be harder to isolate.

IV. THE REFLECTION LOOP

The system’s central design contribution is the analyze node and its conditional edge, which is what elevates the pipeline from a two-step script (search, then summarize) to something with an actual runtime decision in it.

After a search executes, the analyze node issues a second LLM call that asks the model to judge, of its own accumulated search results, whether they constitute “a solid news summary” — with the model instructed to reply with exactly one word first, SUFFICIENT or MORE. The response is parsed with a simple prefix check (response.content.strip().upper().startswith("SUFFI to produce a verdict field on the graph state. The conditional edge leaving analyze then routes to summarize if verdict == "SUFFICIENT" or if the running search count has reached MAX_SEARCHES, and otherwise routes back to search to gather more information before judging again.

The MAX_SEARCHES = 3 cap is the safety property that makes this loop acceptable to run unattended: without it, a topic on which the model persistently judges its own results as insufficient — because the topic is obscure, the search API is degraded, or the model’s judgment is simply miscalibrated — would search indefinitely, at unbounded latency and API

cost. The cap is a static constant rather than an adaptive budget, which we flag explicitly in Section VI as a simplification rather than a considered design decision to prefer static limits.

V. STRUCTURED OUTPUT AND THE MULTIMODAL NODE

Once the reflection loop is satisfied, the `summarize` node does not ask the model to produce a paragraph. Instead, the call is made via `llm.with_structured_output(ResearchReport)`, where `ResearchReport` is a Pydantic model with fixed fields — a headline, a list of bullet points, a list of sources, and an `image_prompt` string. This matters for the same reason structured output matters in most agent-to-application integrations: the consumer of this data is a React frontend that needs to render a headline in one place and a bulleted list in another, and a schema-validated object removes the need for that frontend (or an intermediate parsing layer) to heuristically split an unstructured string into those pieces, and removes an entire class of rendering bugs that would otherwise arise from the model occasionally deviating from an implied-but-unenforced text format.

The `image_prompt` field is then consumed by a distinct `generate_image` graph node, which calls `Pollinations.ai`, a free image-generation service, to produce a cover image for the report. This node is architecturally separate from the summarization call — it is a second model, with a different modality, invoked as its own step in the same graph — which is what we mean by describing the system as multimodal rather than merely text-generating. The `image_prompt` itself is deliberately constrained, at the prompt-authoring level, to describe pure visual scenes with no requested text or labels, a constraint that exists specifically because of a debugging finding described in Section VII: image-generation models of this kind render legible text unreliably, so any attempt to have the cover image itself carry the headline as rendered text produced garbled output, and the fix was to keep the real headline and bullet points as ordinary rendered website text and let the image be purely illustrative.

VI. DEPLOYMENT

The finished Phase 6 graph, wrapped by the Phase 7 FastAPI layer, is packaged with Docker and deployed as three cooperating Azure services: **Azure Container Apps** runs the backend continuously and exposes it at a public address, with the container image itself stored in **Azure Container Registry**; **Azure Blob Storage** persists generated cover images so that they survive a container restart, falling back to local disk if blob storage is not configured; and **Azure Static Web Apps** serves the React frontend’s static Vite build separately, since the frontend does not require an always-running server process the way the backend does. Secrets (the Groq and Tavily API keys) are provisioned as Container App secrets rather than being baked into the container image, so that the built image itself contains no credentials.

Because the deployment was run on a student Azure subscription, the live cloud resources were deliberately

torn down after the demonstration session to control cost; the project’s documentation records the exact `az` CLI command sequence (`az group create`, `az acr create`, `az containerapp env create`, `az containerapp create`, `az storage account create`, `az staticwebapp create`, followed by the image build/push and frontend deploy) needed to reconstitute the deployment from the repository at any time, so that the absence of a currently running instance does not represent a loss of reproducibility.

VII. DEBUGGING STORIES

Three concrete failures encountered while moving from a working local instance to a working cloud deployment are worth recording in detail, because each reflects a class of problem that a purely local development workflow does not surface, and each has a general lesson beyond this specific project.

A. CPU architecture mismatch

The container image built and ran correctly on the development machine (Apple Silicon, an ARM64 architecture) but failed on Azure Container Apps, which runs on x86-64. The Docker image had been built for the wrong target architecture without that being specified explicitly, so a build that appeared to succeed locally produced an artifact that could not execute in the deployment environment. The general lesson is that container builds intended for a different deployment architecture than the build machine’s own must specify their target platform explicitly (e.g., via `docker buildx multi-platform builds`) rather than relying on the build machine’s native architecture by default.

B. A malformed secret

After deployment, the backend could not authenticate against its LLM provider, despite the corresponding API key having been provisioned as a Container App secret. The root cause was a formatting error in how the secret value had been captured — leftover quotation marks included as part of the stored string itself — so the value sent to the provider was not the literal key but the key wrapped in an extra pair of quote characters, which the provider’s authentication check correctly rejected. The fix was to sanitize how the secret value was read and provisioned. The general lesson is that secret values should be validated for exact byte-for-byte equality with the credential as issued, since shell quoting and copy-paste artifacts are a common and easy-to-miss source of authentication failures that manifest identically to a genuinely wrong or revoked key.

C. Stale-tag redeploy caching

An update to the backend code, rebuilt and pushed to the container registry, did not take effect on Azure after redeployment — the running container continued to exhibit the old behavior. The cause was that the same image tag was being reused across the update, and Azure Container Apps’ revision management did not treat a same-tag push as sufficient

grounds to pull and run a new revision. The fix, and the practice adopted going forward, was to assign each build a distinct version tag rather than overwriting a fixed tag such as `latest`. We note explicitly that this is standard practice in production container deployment generally, not a workaround specific to this project’s toolchain.

VIII. LIMITATIONS

The following limitations are drawn directly from the project’s own documentation and are stated candidly, consistent with the project’s own framing of itself as a learning exercise in agentic design patterns rather than a production research tool.

A. Image generation

The system relies on Pollinations.ai, a free-tier image-generation service with no quality or uptime guarantee. As discussed in Section V, such models render legible text unreliably, so all image prompts are constrained to describe pure visual scenes with no requested text, which limits the expressive range of the generated cover image relative to a model with reliable text rendering.

B. No session or conversation memory

Each request to the system is stateless: there is no persistent chat history and no memory of a user’s prior queries across requests, so the system cannot, for example, refine a previous report based on a follow-up question without re-running the entire pipeline from a fresh query.

C. A fixed, non-adaptive search cap

`MAX_SEARCHES = 3` is a static safety limit rather than an adaptive budget informed by, for example, the marginal information gain of each additional search or the cost sensitivity of a given deployment context. It prevents runaway execution but is not tuned per topic or per user.

D. No automated tests

The system was built and verified manually at each of its eight incremental phases rather than through an automated test suite; correctness at each phase was established by direct inspection and manual execution rather than by a durable, re-runnable test artifact.

IX. CONCLUSION AND FUTURE WORK

We have presented a small LangGraph-based research assistant whose central contribution is a bounded reflection loop — a second LLM call that judges the sufficiency of search results and conditionally triggers further search, capped to prevent runaway execution — combined with a Pydantic-validated structured-output contract for its final report and a downstream multimodal image-generation node driven by a text-only, vision-constrained prompt. We have described the system’s incremental, phase-by-phase construction, its containerized deployment to Azure Container Apps, Blob Storage, and Static Web Apps, and three concrete deployment failures — a cross-architecture container build, a malformed secret value, and a

stale-image-tag redeploy — each of which generalizes beyond this specific project to container-based cloud deployment more broadly.

We have been explicit that this is a learning-oriented engineering project rather than a novel research contribution or a production-grade research tool, and its own documented roadmap reflects that framing: replacing manual log-tailing with Azure Monitor / Application Insights; migrating secrets from Container App secrets to a centrally managed, rotatable Key Vault; adding a second LLM provider (e.g., Microsoft Foundry) behind the existing ChatGroq wrapper to empirically validate the claimed provider-agnostic design rather than merely asserting it structurally; and adding automated unit tests per graph node together with an integration test covering the full `search → analyze → summarize → generate_image` path. Realizing that roadmap represents the concrete distance between the current demonstration and a more operationally mature version of the same system.